

Modular Abstraction & Transfer Engine

Problem: Modern AI systems effectively handle context as a single unstructured input which breaks down as interactions grow. Context grows larger unrestrained, becoming difficult to control and slower to process. Even simple information becomes hard to retrieve, responses degrade, and users are forced to restart or continue in another session. The traditional workaround of generating a “turnover script” and passing it into a new session falls short. These scripts are unstructured, lossy, and combine critical information with irrelevant details. There is no clean way to control what is included, enforce constraints, or exclude sensitive data. The core struggle isn’t concerned solely with size; it’s also a matter of shape.

System: In this project, I introduce a modular representation of context. Given an unstructured transcript, the system extracts and organizes information into a fixed set of modules: objective, task state, key decisions, constraints, instructions, preferences, and identity/background. Each module is independently defined and carries metadata including token size, priority level (core versus accessory), privacy sensitivity (shareable, private, replaceable), and inclusion (True or False). This transforms context from a monolithic prompt into a structured object.

Assembly: As a layer sitting over this representation, the system composes context under constraints. In manual mode, the modules can be explicitly included or excluded. In auto mode, modules are selected based on priority, token budget, and privacy constraints, aiming to preserve the minimum sufficient context required to continue a task. Instead of complete and indiscriminate compression, the system enables selective composition.

Outputs: The system produces a full transfer prompt, a compact, budget-constrained prompt, and a modular context packet (JSON). The packet exposes structure, inclusion decisions, and metadata, making context inspectable and reusable rather than opaque.

Implications: As LLMs improve, the bottleneck shifts from model capability to the data and context they operate on. There is clearly room for algorithmic advances, including retrieval-augmented generation, parsing, and more efficient context utilization, but these are inherently research problems. New methods may be readily incorporated, but what we need today is the scaffolding to support them. This project focuses on that system layer.

Instead of viewing context in terms of storage and retrieval, we transition to structure and assembly under constraints. This is especially important in agent workflows, where different steps may call for different subsets of context, instead of passing the entire prompt everywhere. A modular representation allows context to be routed selectively, aligning with Pacific’s idea of a portable, permissioned context layer. An inspectable context layer lays the foundation for managing permissions, enabling agent workflows, and integrating with evolving retrieval and search systems. Rather than replacing existing approaches, it offers a structured layer that can unify and support them. This prototype focuses on reconstructing structured context from unstructured inputs. A natural extension is to maintain this structure incrementally as state evolves, updating only the relevant modules as decisions, constraints, and objectives change. This avoids repeatedly regenerating context from scratch and moves toward a persistent, continuously maintained context layer.